⚠️ Draft Standards Track: ERC

ERC-8004: Trustless Agents

Discover agents and establish trust through reputation and validation

Authors	Marco De Rossi (@MarcoMetaMask), Davide Crapis (@dcrapis) <davide@ethereum.org>, Jordan Ellis <jordanellis@google.com>, Erik Reppel <erik.reppel@coinbase.com>

Created	2025-08-13
Discussion Link	https://ethereum-magicians.org/t/erc-8004-trustless-agents/25098
Requires	EIP-155, EIP-712, EIP-721, EIP-1271

Table of Contents

- Abstract
- Motivation
- Specification
    - Identity Registry
    - Reputation Registry
    - Validation Registry
- Rationale
- Security Considerations
- Copyright

Abstract

This protocol proposes to use blockchains to **discover, choose, and interact with agents across organizational boundaries** without pre-existing trust, thus **enabling open-ended agent economies**.

Trust models are pluggable and tiered, with security proportional to value at risk, from low-stake tasks like ordering pizza to high-stake tasks like medical diagnosis. Developers can choose from different trust models: reputation systems using client feedback, validation via stake-secured re-execution, zero-knowledge machine learning (zkML) proofs, or trusted execution environment (TEE) oracles.

Motivation

Model context protocol (MCP) allows servers to list and offer their capabilities (prompts, resources, tools, and completions), while Agent2Agent (A2A) handles agent authentication, skills advertisement via AgentCards, direct messaging, and complete task-lifecycle

orchestration. However, these agent communication protocols don't inherently cover agent discovery and trust.

To foster an open, cross-organizational agent economy, we need mechanisms for discovering and trusting agents in untrusted settings. This ERC addresses this need through three lightweight registries, which can be deployed on any L2 or on Mainnet as per-chain singletons:

Identity Registry - A minimal on-chain handle based on ERC-721 with URIStorage extension that resolves to an agent's registration file, providing every agent with a portable, censorship-resistant identifier.

Reputation Registry - A standard interface for posting and fetching feedback signals. Scoring and aggregation occur both on-chain (for composability) and off-chain (for sophisticated algorithms), enabling an ecosystem of specialized services for agent scoring, auditor networks, and insurance pools.

Validation Registry - Generic hooks for requesting and recording independent validators checks (e.g. stakers re-running the job, zkML verifiers, TEE oracles, trusted judges).

Payments are orthogonal to this protocol and not covered here. However, examples are provided showing how **x402 payments** can enrich feedback signals.

Specification

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in RFC 2119 and RFC 8174.

Identity Registry

The Identity Registry uses ERC-721 with the URIStorage extension for agent registration, making **all agents immediately browsable and transferable with NFTs-compliant apps**. Each agent is uniquely identified globally by:

- *agentRegistry*: A colon-separated string `{namespace}:{chainId}:{identityRegistry}` (e.g., `eip155:1:0x742...`) where:
  - *namespace*: The chain family identifier (`eip155` for EVM chains)
  - *chainId*: The blockchain network identifier
  - *identityRegistry*: The address where the ERC-721 registry contract is deployed
- *agentId*: The ERC-721 tokenId assigned incrementally by the registry

Throughout this document, *tokenId* in ERC-721 is referred to as *agentId* and *tokenURI* in ERC-721 is referred to as *agentURI*. The owner of the ERC-721 token is the owner of the agent and can transfer ownership or delegate management (e.g., updating the registration file) to operators, as supported by `ERC721URIStorage`.

Agent URI and Agent Registration File

The *agentURI* MUST resolve to the agent registration file. It MAY use any URI scheme such as `ipfs://` (e.g., `ipfs://cid`), `https://` (e.g., `https://example.com/agent3.json`), or a base64-encoded `data:` URI (e.g., `data:application/json;base64,eyJ0eXBlIjoi...`) for fully on-chain metadata. When the registration uri changes, it can be updated with *setAgentURI()*.

The registration file MUST have the following structure:

```
{
  "type": "https://eips.ethereum.org/EIPS/eip-8004#registration-v1",
  "name": "myAgentName",
  "description": "A natural language description of the Agent, which MAY incl
  "image": "https://example.com/agentimage.png",
  "services": [
   {
      "name": "web",
      "endpoint": "https://web.agentxyz.com/"
    },
    {
      "name": "A2A",
      "endpoint": "https://agent.example/.well-known/agent-card.json",
      "version": "0.3.0"
    },
    {
      "name": "MCP",
      "endpoint": "https://mcp.agent.eth/",
      "version": "2025-06-18"
    },
    {
      "name": "OASF",
      "endpoint": "ipfs://{cid}",
      "version": "0.8", // https://github.com/agntcy/oasf/tree/v0.8.0
      "skills": [], // OPTIONAL
      "domains": [] // OPTIONAL
    },
    {
      "name": "ENS",
      "endpoint": "vitalik.eth",
      "version": "v1"
    },
    {
      "name": "DID",
      "endpoint": "did:method:foobar",
      "version": "v1"
    },
    {
      "name": "email",
```

```
      "endpoint": "mail@myagent.com"
    }
  ],
  "x402Support": false,
  "active": true,
  "registrations": [
    {
      "agentId": 22,
      "agentRegistry": "{namespace}:{chainId}:{identityRegistry}" // e.g. eip
    }
  ],
  "supportedTrust": [
    "reputation",
    "crypto-economic",
    "tee-attestation"
  ]
}
```

The *type*, *name*, *description*, and *image* fields at the top SHOULD ensure compatibility with ERC-721 apps. The number and type of *endpoints* are fully customizable, allowing developers to add as many as they wish. The *version* field in endpoints is a SHOULD, not a MUST.

Agents MAY advertise their endpoints, which point to an A2A agent card, an MCP endpoint, an ENS agent name, DIDs, or the agent's wallets on any chain (even chains where the agent is not registered).

Endpoint Domain Verification (Optional)

Since endpoints can point to domains not controlled by the agent owner, an agent MAY optionally prove control of an HTTPS endpoint-domain by publishing `https://{endpoint-domain}/.well-known/agent-registration.json` containing at least a `registrations` list (or the full agent registration file). Users MAY treat the endpoint-domain as verified if the file is reachable over HTTPS and includes a `registrations` entry whose `agentRegistry` and `agentId` match the on-chain agent; if the endpoint-domain is the same domain that serves the agent's primary registration file referenced by `agentURI`, this additional check is not needed because domain control is already demonstrated there.

Agents SHOULD have at least one registration (multiple are possible), and all fields in the registration are mandatory. The *supportedTrust* field is OPTIONAL. If absent or empty, this ERC is used only for discovery, not for trust.

On-chain metadata

The registry extends ERC-721 by adding `getMetadata(uint256 agentId, string metadataKey)` and `setMetadata(uint256 agentId, string metadataKey, bytes metadataValue)` functions for optional extra on-chain agent metadata:

```
function getMetadata(uint256 agentId, string memory metadataKey) external vie
function setMetadata(uint256 agentId, string memory metadataKey, bytes memory
```

When metadata is set, the following event is emitted:

```
event MetadataSet(uint256 indexed agentId, string indexed indexedMetadataKey,
```

The key `agentWallet` is reserved and cannot be set via `setMetadata()` or during `register()` (including the metadata array overload). It represents the address where the agent receives payments and is initially set to the owner's address. To change it, the agent owner must prove control of the new wallet by providing a valid EIP-712 signature for EOAs or ERC-1271 for smart contract wallets—by calling:

```
function setAgentWallet(uint256 agentId, address newWallet, uint256 deadline,
```

To read and clear the currently set wallet, the following functions are exposed:

```
function getAgentWallet(uint256 agentId) external view returns (address)
function unsetAgentWallet(uint256 agentId) external
```

When the agent is transferred, `agentWallet` is automatically cleared (effectively resetting it to the zero address) and must be re-verified by the new owner.

Registration

New agents can be minted by calling one of these functions:

```
struct MetadataEntry {
string metadataKey;
bytes metadataValue;
}

function register(string agentURI, MetadataEntry[] calldata metadata) externa

function register(string agentURI) external returns (uint256 agentId)

// agentURI is added later with setAgentURI()
function register() external returns (uint256 agentId)
```

This emits one Transfer event, one MetadataSet event for the reserved `agentWallet` key, one MetadataSet event for each additional metadata entry (if any), and

```
event Registered(uint256 indexed agentId, string agentURI, address indexed ow
```

Update agentURI

The agentURI can be updated by calling the following function, which emits a URIUpdated event:

```
function setAgentURI(uint256 agentId, string calldata newURI) external

event URIUpdated(uint256 indexed agentId, string newURI, address indexed upda
```

If the owner wants to store the entire registration file on-chain, the *agentURI* SHOULD use a base64-encoded data URI rather than a serialized JSON string:

```
data:application/json;base64,eyJ0eXBlIjoi...
```

Reputation Registry

When the Reputation Registry is deployed, the *identityRegistry* address is set via `initialize(address identityRegistry_)` and publicly visible by calling:

```
function getIdentityRegistry() external view returns (address identityRegistr
```

The feedback given by a *clientAddress* to an agent consists of a signed fixed-point *value* (`int128`) and its *valueDecimals* (`uint8`, 0-18), plus optional *tag1* and *tag2* (left to developers' discretion to provide maximum on-chain composability and filtering), an *endpoint* URI, a file URI pointing to an off-chain JSON containing additional information, and its KECCAK-256 file hash to guarantee integrity. We suggest using IPFS or equivalent services to make feedback easily indexed by subgraphs or similar technologies. For IPFS URIs, the hash is not required. All fields except *value* and *valueDecimals* are OPTIONAL, so the off-chain file is not required and can be omitted.

Giving Feedback

New feedback can be added by any *clientAddress* calling:

```
function giveFeedback(uint256 agentId, int128 value, uint8 valueDecimals, str
```

The *agentId* must be a validly registered agent. The *valueDecimals* MUST be between 0 and 18. The feedback submitter MUST NOT be the agent owner or an approved operator for *agentId*. *tag1*, *tag2*, *endpoint*, *feedbackURI*, and *feedbackHash* are OPTIONAL.

Where provided, *feedbackHash* is the KECCAK-256 hash (`keccak256`) of the content referenced by *feedbackURI*, enabling verifiable integrity for non-content-addressed URIs. For IPFS (or other content-addressed URIs), *feedbackHash* is OPTIONAL and can be omitted (e.g., set to `bytes32(0)`).

If the procedure succeeds, an event is emitted:

```
event NewFeedback(uint256 indexed agentId, address indexed clientAddress, uin
```

The feedback fields *value*, *valueDecimals*, *tag1*, *tag2*, and *isRevoked* are stored in the contract storage along with the feedbackIndex (a 1-indexed counter of feedback submissions that *clientAddress* has given to *agentId*). The fields *endpoint*, *feedbackURI*, and *feedbackHash* are emitted but are not stored. This exposes reputation signals to any smart contract, enabling on-chain composability.

When the feedback is given by an agent (i.e., the client is an agent), the agent SHOULD use the address set in the on-chain optional `agentWallet` metadata as the clientAddress, to facilitate reputation aggregation.

Examples of `value` / `valueDecimals`

tag1	What it measures	Example human value	`value`	`valueDecimals`
`starred`	Quality rating (0-100)	`87/100`	`87`	`0`
`reachable`	Endpoint reachable (binary)	`true`	`1`	`0`
`ownerVerified`	Endpoint owned by agent owner (binary)	`true`	`1`	`0`
`uptime`	Endpoint uptime (%)	`99.77%`	`9977`	`2`
`successRate`	Endpoint success rate (%)	`89%`	`89`	`0`
`responseTime`	Response time (ms)	`560ms`	`560`	`0`
`blocktimeFreshness`	Avg block delay (blocks)	`4 blocks`	`4`	`0`
`revenues`	Cumulative revenues (e.g., USD)	`$560`	`560`	`0`
`tradingYield` (`tag2` = `day, week, month, year`)	Yield	`-3,2%`	`-32`	`1`

Off-Chain Feedback File Structure

The OPTIONAL file at the URI could look like:

```
{
  // MUST FIELDS
  "agentRegistry": "eip155:1:{identityRegistry}",
  "agentId": 22,
  "clientAddress": "eip155:1:{clientAddress}",
  "createdAt": "2025-09-23T12:00:00Z",
  "value": 100,
  "valueDecimals": 0,

  // ALL OPTIONAL FIELDS
  "tag1": "foo",
  "tag2": "bar",
  "endpoint": "https://agent.example.com/GetPrice",

  "mcp": { "tool": "ToolName" }, // or: { "prompt": "PromptName" } / { "resou

  // A2A: see "Context Identifier Semantics" and Task model in the A2A specif
  "a2a": {
    "skills": ["as-defined-by-A2A"], // e.g., AgentSkill identifiers
    "contextId": "as-defined-by-A2A",
    "taskId": "as-defined-by-A2A"
  },

  "oasf": {
    "skills": ["as-defined-by-OASF"],
    "domains": ["as-defined-by-OASF"]
  },

  "proofOfPayment": { // this can be used for x402 proof of payment
          "fromAddress": "0x00...",
          "toAddress": "0x00...",
          "chainId": "1",
          "txHash": "0x00..."
   },

 // Other fields
  " ... ": { " ... " } // MAY
}
```

Revoking Feedback

clientAddress can revoke feedback by calling:

```
function revokeFeedback(uint256 agentId, uint64 feedbackIndex) external
```

This emits:

```
event FeedbackRevoked(uint256 indexed agentId, address indexed clientAddress,
```

Appending Responses

Anyone (e.g., the *agentId* showing a refund, any off-chain data intelligence aggregator tagging feedback as spam) can call:

```
function appendResponse(uint256 agentId, address clientAddress, uint64 feedba
```

Where *responseHash* is the KECCAK-256 file hash of the *responseURI* file content to guarantee integrity. This field is not required for IPFS URIs.

This emits:

```
event ResponseAppended(uint256 indexed agentId, address indexed clientAddress
```

Read Functions

```
function getSummary(uint256 agentId, address[] calldata clientAddresses, stri
// agentId and clientAddresses are mandatory; tag1 and tag2 are optional filt
// clientAddresses MUST be provided (non-empty); results without filtering by

function readFeedback(uint256 agentId, address clientAddress, uint64 feedback

function readAllFeedback(uint256 agentId, address[] calldata clientAddresses,
// agentId is the only mandatory parameter; others are optional filters. Revo

function getResponseCount(uint256 agentId, address clientAddress, uint64 feed
// agentId is the only mandatory parameter; others are optional filters.

function getClients(uint256 agentId) external view returns (address[] memory)

function getLastIndex(uint256 agentId, address clientAddress) external view r
```

We expect reputation systems around reviewers/clientAddresses to emerge. **While simple filtering by reviewer (useful to mitigate spam) and by tag are enabled on-chain, more complex reputation aggregation will happen off-chain**.

Validation Registry

This registry enables agents to request verification of their work and allows validator smart contracts to provide responses that can be tracked on-chain. Validator smart

contracts could use, for example, stake-secured inference re-execution, zkML verifiers or TEE oracles to validate or reject requests.

When the Validation Registry is deployed, the *identityRegistry* address is set via `initialize(address identityRegistry_)` and is visible by calling `getIdentityRegistry()`, as described above.

Validation Request

Agents request validation by calling:

```
function validationRequest(address validatorAddress, uint256 agentId, string
```

This function MUST be called by the owner or operator of *agentId*. The *requestURI* points to off-chain data containing all information needed for the validator to validate, including inputs and outputs needed for the verification. The *requestHash* is a commitment to this data (`keccak256` of the request payload) and identifies the request. All other fields are mandatory.

A ValidationRequest event is emitted:

```
event ValidationRequest(address indexed validatorAddress, uint256 indexed age
```

Validation Response

Validators respond by calling:

```
function validationResponse(bytes32 requestHash, uint8 response, string respo
```

Only *requestHash* and *response* are mandatory; *responseURI*, *responseHash* and *tag* are optional. This function MUST be called by the *validatorAddress* specified in the original request. The *response* is a value between 0 and 100, which can be used as binary (0 for failed, 100 for passed) or with intermediate values for validations with a spectrum of outcomes. The optional *responseURI* points to off-chain evidence or audit of the validation, *responseHash* is its commitment (in case the resource is not on IPFS), while *tag* allows for custom categorization or additional data.

validationResponse() can be called multiple times for the same *requestHash*, enabling use cases like progressive validation states (e.g., “soft finality” and “hard finality” using *tag*) or updates to validation status.

Upon successful execution, a *ValidationResponse* event is emitted with all function parameters:

```
event ValidationResponse(address indexed validatorAddress, uint256 indexed ag
```

The contract stores *requestHash*, *validatorAddress*, *agentId*, *response*, *responseHash*, *lastUpdate*, and *tag* for on-chain querying and composability.

Read Functions

```
function getValidationStatus(bytes32 requestHash) external view returns (addr

//Returns aggregated validation statistics for an agent. agentId is the only
function getSummary(uint256 agentId, address[] calldata validatorAddresses, s

function getAgentValidations(uint256 agentId) external view returns (bytes32[

function getValidatorRequests(address validatorAddress) external view returns
```

Incentives and slashing related to validation are managed by the specific validation protocol and are outside the scope of this registry.

Rationale

- **Agent communication protocols**: MCP and A2A are popular, and other protocols could emerge. For this reason, this protocol links from the blockchain to a flexible registration file including a list where endpoints can be added at will, combining AI primitives (MCP, A2A) and Web3 primitives such as wallet addresses, DIDs, and ENS names.
- **Feedback**: The protocol combines the leverage of nomenclature already established by A2A (such as tasks and skills) and MCP (such as tools and prompts) with complete flexibility in the feedback signal structure.
- **Gas Sponsorship**: Since clients don't need to be registered anymore, any application can implement frictionless feedback leveraging EIP-7702.
- **Indexing**: Since feedback data is saved on-chain and we suggest using IPFS for full data, it's easy to leverage subgraphs to create indexers and improve UX.
- **Deployment**: We expect the registries to be deployed with singletons per chain. Note that an agent registered and receiving feedback on chain A can still operate and transact on other chains. Agents can also be registered on multiple chains if desired.

Security Considerations

- Sybil attacks are possible, inflating the reputation of fake agents. The protocol's contribution is to make signals public and use the same schema. We expect many players to build reputation systems, for example, trusting or giving reputation to reviewers (and therefore filtering by reviewer, as the protocol already enables).

- On-chain pointers and hashes cannot be deleted, ensuring audit trail integrity
- Validator incentives and slashing are managed by specific validation protocols
- While this ERC cryptographically ensures the registration file corresponds to the on-chain agent, it cannot cryptographically guarantee that advertised capabilities are functional and non-malicious. The three trust models (reputation, validation, and TEE attestation) are designed to support this verification need

Copyright

Copyright and related rights waived via CC0.

Citation

Please cite this document as:

Marco De Rossi (@MarcoMetaMask), Davide Crapis (@dcrapis) <davide@ethereum.org>, Jordan Ellis <jordanellis@google.com>, Erik Reppel <erik.reppel@coinbase.com>, "ERC-8004: Trustless Agents [DRAFT]," *Ethereum Improvement Proposals*, no. 8004, August 2025. Available: https://eips.ethereum.org/EIPS/eip-8004.